

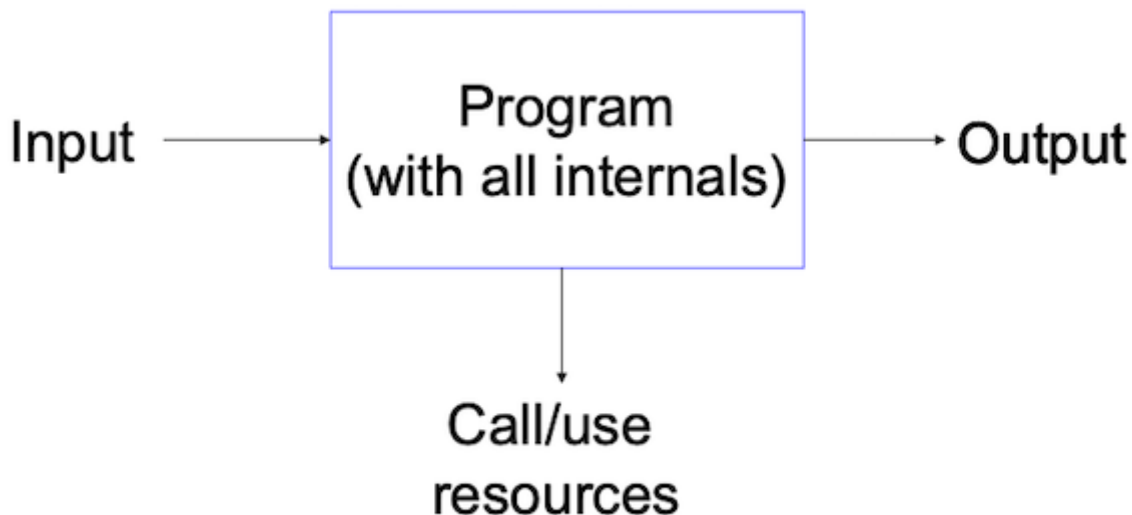
Secure Software Design

So Far: Focus on Internals

- Avoid memory vulnerabilities.
- Proper data/input validation.
- Handling errors and exceptions.
- Language-specific security considerations:
 - ~~Object orientation~~.
 - ~~Concurrency~~.

The Big Picture

- not to produce **outputs** that make other programs downstream vulnerables.
- **input validation**
- **resources**: what assumptions to make



Schema after David A. Wheeler.

Design Principles

Key Principles

- Saltzer & Schroeder: [Paper](#)

Fail-safe defaults

- When a system fails, it should default to a secure state.
 - => This means that, in the event of an error or failure (e.g., software bug, crash, network issue), the system should not allow access or actions unless explicitly allowed.
- Confidentiality/integrity => **fail closed**.
- Availability => **fail open**.

Fail closed means:

- If the system cannot decide whether to allow or deny access due to a failure, it should deny access.

Example 1:

- A firewall loses its configuration or crashes. To protect data confidentiality and integrity, it should block all traffic (fail closed), rather than allow it.

Fail open means:

- If a failure occurs, the system continues operating, even if that means relaxing some controls.

Example 2:

- An emergency door system in a hospital: if the power goes out, the doors should unlock (fail open) to let people out, even if it means compromising some security.

Property	Failure Behaviour	Reason
Confidentiality	Fail closed	Don't expose data
Integrity	Fail closed	Don't allow unverified changes
Availability	Fail open	Keep services running if possible

Economy of mechanism

- **Keep it simple.**
- Make audit/verification/testing easier.
- Complexity introduces errors and unwanted execution paths.
- **Simplicity reduces the likelihood of errors**

Least common mechanism

- **Minimize unwanted interactions between components.**
- Reduces risk of security failures affecting unrelated features.
- **sharing mechanisms (like memory, files, or network ports) creates potential points of vulnerability.**

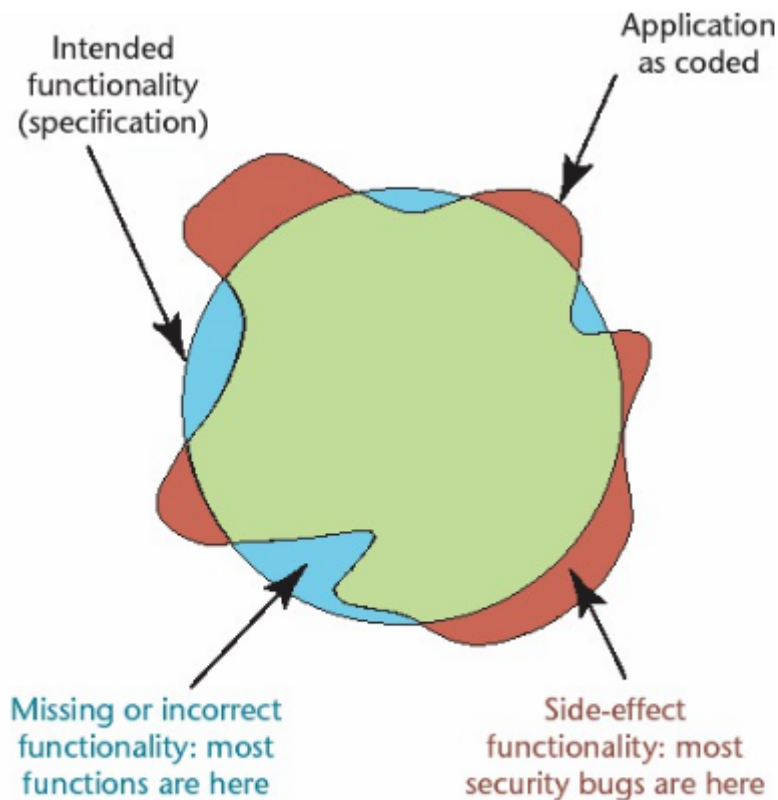
Least privilege

- **Grant the minimal permissions necessary.**
- Programs can drop privileges once they have accomplished their security actions.
- **Example:** **Need-to-know** basis in military security.

Separation of privilege

- **Protect critical actions with independent mechanisms.**
- **Example:** Two-factor authentication.

Simple Design Helps



H. Thompson, Why Security Testing Is Hard, IEEE S&P, 2003.

Minimize Privilege

Methods for Minimizing Privilege

- **Type/strength** of granted privileges.
- **Duration** of privilege activation.
- **Amount of code** executed with privilege.
- **Amount of Data** accessible to privileged execution.

Minimize the type of privilege

- Simplistic view:
 - high privilege is root/admin access
- But: root/admin can do **everything**
 - limit to what is actually needed
- This is an issue of system configuration
- Create roles for access to resources
 - limit privilege to requirements of role

Unix Access Control: Groups and Roles

- **File permissions:** Based on **owner**, **group**, **others**.
- **Groups:** More granular access than a single root user.
- **Examples:** **root**, **daemon**, **sys**, **mail**, **disk**, **audio**.

```

nestorcatano@Nestors-Mac-mini week-06 % ls -la *
-rw-r--r--@ 1 nestorcatano  staff    3816 Apr 24  2023 W204-summary-week06.txt
-rw-r--r--@ 1 nestorcatano  staff  3927114 Feb 27  2020 saltzer_schroeder_secure_design_principles.pdf
-rw-r--r--@ 1 nestorcatano  staff    78982 Feb 27  2020 secure_design_principles_01.pdf
-rw-r--r--@ 1 nestorcatano  staff    61259 Feb 27  2020 secure_design_principles_02.pdf
-rw-r--r--@ 1 nestorcatano  staff  19074016 Feb 27  2020 ucb-mics_6_1_1_lecture_slides_secure_design_overview_360.mp4
-rw-r--r--@ 1 nestorcatano  staff   5639702 Feb 27  2020 ucb-mics_6_1_2_lecture_slides_minimize_privileges_360.mp4.crdownload.mp4
-rw-r--r--@ 1 nestorcatano  staff  36200460 Feb 27  2020 ucb-mics_6_1_3_lecture_slides_avoid_race_conditions_360.mp4

```

Unix **setuid** Programs

- A **setuid** program is a Unix/Linux executable that runs with the privileges of the **file's owner**, rather than the user who launched it. This is enabled by setting the set-user-ID (**setuid**) permission bit on the file.
- It changes the **effective user ID**
- **Example:** **/usr/bin/passwd** allows users to change their own passwords.

```

$ ls -l /usr/bin/passwd
-rwsr-xr-x 1 root root 54256 Apr 10 12:34 /usr/bin/passwd

```

passwd allows users to update their own password

Unix **setgid** Programs

A **setgid** (**set group ID**) program is similar to a **setuid** program, but instead of changing the **effective user ID (EUID)** during execution, it changes the **effective group ID (EGID)** to that of the file's group owner

```

$ ls -l /usr/bin/wall
-r-xr-sr-x 1 root tty 38984 Apr 10 12:34 /usr/bin/wall

```

wall (used to send messages to all users) runs with group **tty**, so it can write to user terminals, which a normal user otherwise couldn't.

Risks of **setuid** Programs

- Setuid programs can be flawed like others
 - **> 350 CVE entries** for **setuid**
- If a vulnerability (e.g., buffer overflow) is exploited, an attacker could exploit it to gain root access.
- Programs that trust environment variables like **PATH**, **LD_LIBRARY_PATH**, or **IFS** can be tricked into executing malicious code.
- If the program does not drop root privileges after completing privileged tasks, subsequent actions may be dangerous.
- Functions like **gets()**, **sprintf()**, or poor file handling can introduce classic vulnerabilities like stack smashing or symlink attacks.
- **Race conditions:** **time-of-check-to-time-of-use (TOCTOU)** bugs can be exploited to substitute malicious files during execution.

setuid Mechanism

- A process has several types of user ID
 - **Real user ID**: the actual ID of the user that has started the process
 - **Effective user ID**: gives the actual rights available to the process
 - **Saved user ID** (in most newer versions): stores initial value of the real user ID reference point when switching between IDs/permissions

Manipulating privilege

- A process can use the **setuid** call to change these **IDs** and run with corresponding privileges
- A root process (**euid** = 0) can set its **Real user ID** to any value
- A non-root process can call **setuid(newID)** only if **newID** is one of the three existing IDs (**real**, **effective**, **saved**)
- **setuid(newID)** sets all three user IDs (the change is irreversible)

Dropping privileges

- If possible, drop privilege completely, using **setuid()**, once needed task is done
 - *Example*: web server needs root to connect to port **80** but can then drop privilege
- Otherwise, use **seteuid()** to switch between high- and low-privilege code (use high privilege only where needed)

Example Scenario

Let's say **user bob** (**UID 1000**) runs a binary owned by root that has the **setuid** bit set:

```
ls -l /usr/bin/passwd
-rwsr-xr-x 1 root root ...
```

When bob runs **/usr/bin/passwd**, here's what happens:

RUID = 1000 (bob)

EUID = 0 (root, because of **setuid**)

SUID = 0 (saved root privileges for later)

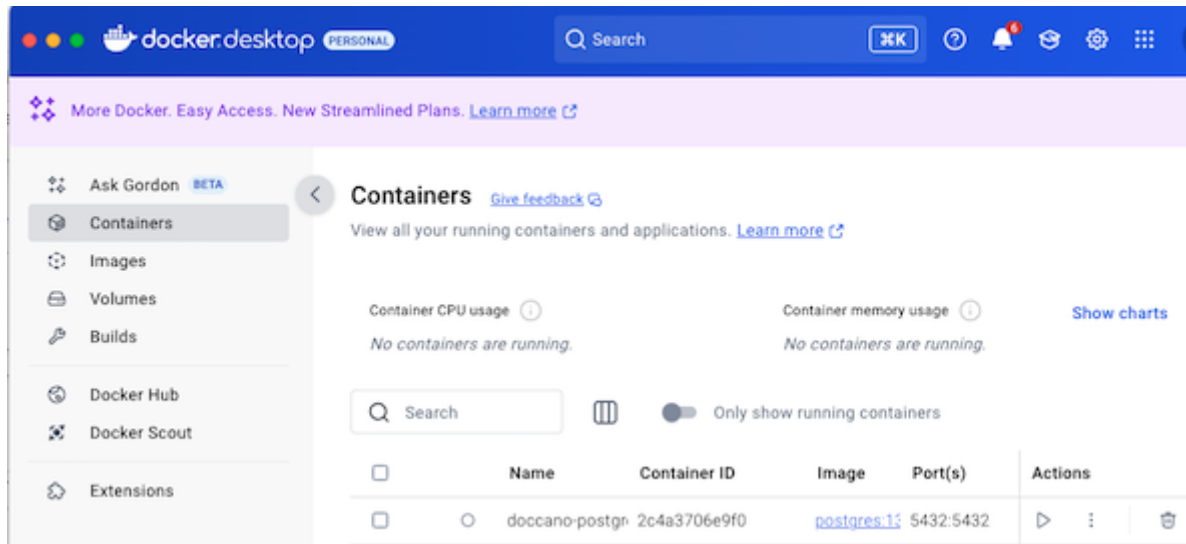
- The process can now access files only root can access (like **/etc/shadow**) thanks to its **EUID** being 0.
- If it wants to perform unprivileged actions securely, it can drop its **EUID** to **1000** temporarily, then later restore it from the **SUID**.

chroot and chroot jail

- The **chroot()** system call (short for **change root**) is used to change the apparent root directory (**/**) for the current running process and its children.
- This creates a **sandboxed environment**, often called a **chroot jail**, isolating the process from the rest of the system.

Minimize accessible files

- Can use `chroot()` to limit file system part accessible to program
- Set up directory with minimal number of needed files (including any libraries needed by program)
- Ensure that directory is never user accessible (cannot be tampered with)
- Program should not have root rights (avoid escape from `chroot jail`)



Purpose of `chroot`

Security: Limit what files a process can see/access.

Testing/Recovery: Run tools in a separate filesystem (e.g., during system recovery).

Build environments: Compile software in isolated trees.

Minimal containers: Basic process isolation (precursor to full containerisation like Docker).

Basic usage of `chroot`

```
sudo chroot /my/chroot/environment /bin/bash
```

- Changes the root directory to `/my/chroot/environment`.
- Launches a shell inside that environment.
- Inside, `/` refers to `/my/chroot/environment`, and the rest of the system is hidden.

```
#include <unistd.h>
#include <stdio.h>

int main() {
    if (chroot("/my/chroot/environment") != 0) {
        perror("chroot failed");
        return 1;
    }
    chdir("/"); // change to the new root directory
```

```
// Process now runs in chroot jail
return 0;
}
```

Avoid Race Conditions

Concurrency Issues

- **Atomicity** not guaranteed by default
- **Race**: processes access shared resource
- **Ensure**: two steps produce correct result even if interrupted by another process
- **Interference** may happen with both trusted and untrusted processes
 - non-atomicity
 - deadlock/livelock

Check and Use Atomically

- Secure programs often need to
 - determine if a request should be granted
 - act on that request
- If the two steps are not atomic, we might have a **time-of-check to time-of-use** race condition
- **Violates complete mediation principle**

Example: TOCTTOU Race in File Access

```
if (access("file.txt", R_OK) == 0) {
    int fd = open("file.txt", O_RDONLY); // Unsafe!
}
```

- **Risk**: `file.txt` could be replaced between `access()` and `open()`.
- **Solution**: Use file descriptors instead of filenames.

Safer File Access

```
int fd = open("file.txt", O_RDONLY);
if (fd != -1) {
    // Safe operations on fd
}
```

Temporary Files

- **Problem**: `/tmp` directory shared by all
- An attacker should not be able to read or write from a temporary file created by another program.
- Can be used by attacker to create links

- Must ensure file does not already exist
 - check before does not work (TOCTTOU)
 - **unique** file name runs risk of guessing
- Risks interfering with temp file cleaners
- **tmpfile()** function creates temporary file and returns open stream to it

Temporary File Security

- **Problem:** `/tmp/` is shared by all users.
- **Unsafe approach:**

```
char tmpfile[] = "/tmp/mytempXXXXXX";  
mktemp(tmpfile); // Unsafe, filename could be guessed!
```

- **Safer approach:**

```
int fd = mkstemp(tmpfile); // Creates and opens a unique temp file
```

Secure Environment

Environment Variables

- **OS environment variables** can **affect program behavior**:
 - `$HOME`, `$PATH`, `$SHELL`, `$TZ`, etc.
- By default, inherited from parent process
 - But can be explicitly **set for child** (alongside program arguments)
 - **Danger:** Malicious users can manipulate these variables to exploit programs.
- **Unix:** variants of `exec()` calls
- Program environment: more than just variables (e.g., open file descriptors)

Danger: Malicious Environment

- Environment designed for program control
 - `=>` can be abused
- **Example:** `$PATH` environment variable
 - `=` directories searched for executables
- If set to attacker-controlled directory, can force execution of malicious programs (e.g., malicious `ls` instead of usual listing)
- Having current directory first in `PATH`: never a good idea

Example: Exploiting `$PATH`

```
export PATH=/malicious_dir:$PATH  
ls # Executes malicious 'ls' instead of system 'ls'
```

- **Prevention:** Never have `.` (current directory) first in `$PATH`.

Secure Environment Setup

- **Sanitize environment for privileged programs:**
 - Remove dangerous variables (`SHELL`, `IFS`).
 - Use minimal standard environment settings.
 - Ensure open file descriptors are secure (`stdin`, `stdout`, `stderr`).

Secure Interfacing

Secure Resource Usage

- Programs **must interact securely** with:
 - **Libraries** (avoid vulnerable dependencies).
 - **Files** (ensure correct access control).
 - **External modules** (verify API usage).

Call Only Secure Libraries

- Not all code is designed for security
- Know library docs and intended purpose
- Consider implementation language, size of code, stability, bug/vulnerability reports
- Remember: economy of mechanism
 - ideally is small, simple, special-purpose
- If needed, consider writing your own
 - Try to keep interface compatible

Call Libraries, Not Programs

- Be aware if library eventually calls shell or other executables (through `system()`, etc.)
- Program interfaces: complex, expressive, intended for humans
 - Often counter to economy of mechanism
 - Potential for misunderstanding/misuse
 - Larger attack surface
 - Special danger: debugging features

Call Only with Valid Values

- Not all libraries implement validation or handle all corner cases correctly
- Defensive: ensure that all parameters have safe values
 - Whitelisting, not blacklisting
 - Special care: control/escape/meta-chars, command injection
 - Problematic filenames, wildcards, and so on

Check all possible errors

- Error handling is itself highly error prone
- Omission errors if:
 - Only some error codes are checked
 - There is no catch-all error branch
- Specific security concerns: syscalls (direct or through standard library calls)
- All syscalls have resource limitations (possibly affected from the outside)
- Can result in denial-of-service attacks